



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

计算机科学与技术

计算机学院 胡思康

skhu@163.com

关于考试

- ❖ 时间: **2024年1月12日上午10:10-12:10(第二十一周)**
- ❖ 地点: 良乡文萃H机房 (机考)
- ❖ 题型:
 - 选择题;
 - 程序填空题;
 - 编程题(3道编程题)。
- ❖ **AB卷**
- ❖ 编程题可以多次提交, **以最后一次提交的程序为准**。选择题要注意**“确认提交”**
- ❖ 考试时不可以上网、不可以用手机、不可以安装软件
- ❖ 需要掌握的IDE (DEV C++)



关于复习

基础知识部分

- 大学计算机（面广，简单）
- C前几章基础知识仍渗透在后续内容中。
- 将做过的看几遍，能用“学术”的语言解释答案。

将做过的题目分类

- 数制、硬件、软件；
- 指针与字符串类的
- 指针与数组类的；
- 函数的概念类题目；
- 变量存储特性类；
- 编译预处理有关的；
- 二维数组的各种地址表达式或元素值

考试复习建议



仔细看题

编程题部分

总结自己常错的：

- scanf 缺少 &
- 循环结构，选择结构缺少 { }
- 循环边界表达不准，所以循环次数不对
- 数组下标越界
- 变量没初始化就使用了
- 输入输出格式与题目用例给的不一致
- 缺少最后的字符串结束标记



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



二进制与计算机

计算机中使用二进值，数据的最小单位是位bit，两个可能的取值0和1。

电位高与低：开关接通与断开，用0，1描述两个稳定的状态。

电子计算机的字长代表了CPU一次存储、处理、传递信息的位数。

- 32-bit 计算机
- 64-bit 计算机



数制和信息编码

- 基数：数制中所含数字符号的个数。
- 权：数制中每一个固定位置对应的单位值。
- 常用数制：二、八、十、十六进制。
- 不同数制之间的转换
 - R进制 $\rightarrow$ 十进制：按权展开。
 - 十进制 $\rightarrow$ R进制：注意整数与小数转换的区别（除基取余法，乘基取整法）
 - 二、八、十六进制之间转换：注意自小数点开始分别向左或向右划分（3位一组或4位一组）



数制和信息编码

- **无符号数和带符号数的区别。**
 - 无符号数的所有位均用来表示数值。
 - 带符号数最高位为符号位，0表示正数，1表示负数。
- **带符号数：原码、补码。**
 - 真值用原码、补码表示：注意正数和负数的区别。
 - 原码和补码表示范围不同。
 - 0 的表示形式。



无符号数与带符号数（原码、补码、反码）

- 相同位数（如8位二进制）原码和补码表示范围不同。
 - 原码表示范围-127~127,
 - 补码表示范围-128~127。
- 原码有两种0的表示形式，+0表示为00000000，-0表示为10000000
- 补码只有唯一一种0的表示形式，00000000
 - 如二进制代码10000000，原码时表示-0，补码时表示-128，反码时表示-127。

数制和信息编码

- 在计算机中整型数用定点数表示，实型数用浮点数表示。
- 假设定点数和浮点数的字长相同。浮点表示法所能表示的数值范围将远大于定点数。
- 浮点数阶码为定点整数，尾数为定点小数，阶码的位数确定表示范围，尾数的位数确定精度。
 - 以字长16位为例，定点整数(补码)为-32768~32767，
即 $-2^{15} \sim 2^{15}-1$ ，浮点数(阶码部分6位，尾数部分10位)
最大值 $\approx 2^{31}$ ，其中阶码最高位是符号位
- ASCII码和逻辑运算



数制和信息编码

- 汉字编码：输入码、交换码、机内码、字型码
 - 输入码（外码）：五笔、拼音等
 - 交换码（国标码）与机内码的转换
 - 汉字内码和字型码的存储量：注意内码（两字节），字型码与点阵大小有关
 - 字形码字库
- 机内码 = 国标码 + 8080H
- 汉字机内码和交换码（国标码）均为两字节。
- 汉字字型码16×16点阵，24×24点阵，32×32点阵等



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



计算机硬件系统基础

➤ 计算机系统 = 硬件系统 + 软件系统

➤ 计算机硬件系统组成

➤ 冯·诺依曼思想：硬件由五大基本部件组成、采用二进制、**存储程序原理**。

➤ 五大部件：输入设备、输出设备、存储器、运算器、控制器

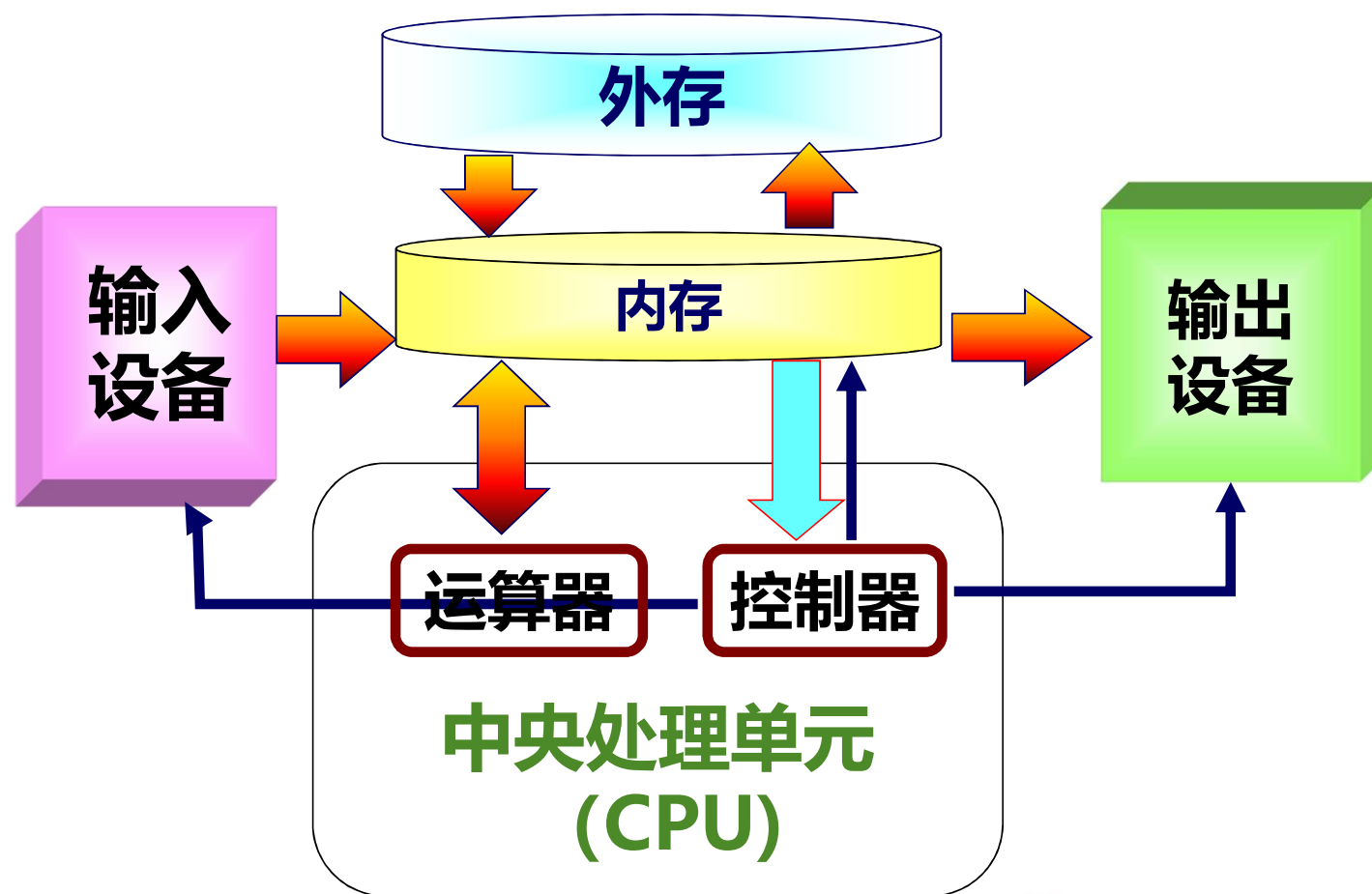
➤ **CPU = 运算器 + 控制器**

➤ CPU、主机、外设等性能参数



计算机硬件系统基础

➡ data
➡ address
➡ control



关于内存

(1) 内存存储当前CPU正在处理的程序和数据。

(4) 内存以字节为单位编址

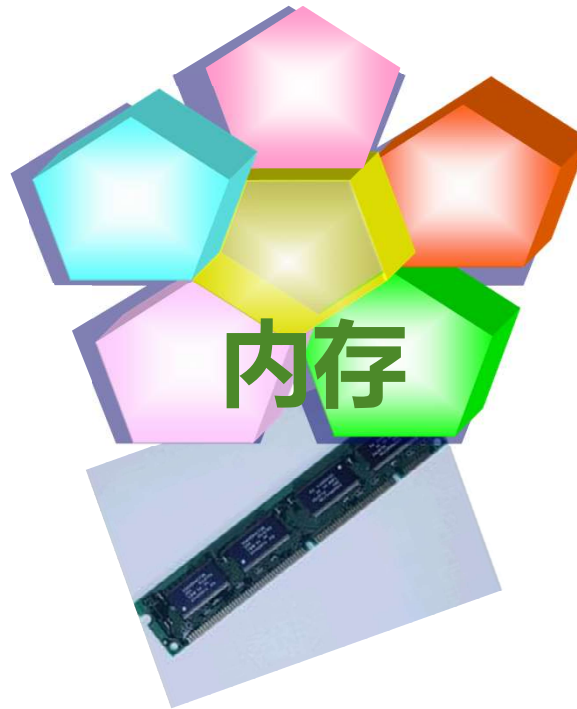
1字节=8位

1024字节=1K

1024K=1M

1024M=1G

1024G=1T



(2) 内存有只读存储ROM, 随机存储RAM和CMOS等。

(3) RAM上的数据是临时的。

- 地址线的数目K与存储空间N的关系: $N=2^K$
- 存储器的地址线根数(K)决定了它的地址编号范围



操作系统基础

➤ 软件系统

- 软件：程序、数据和文档的集合
- 软件系统=系统软件+应用软件
- 系统软件：操作系统、**语言翻译程序**、辅助程序等
- 应用软件：为解决某类实际问题而开发的程序
- 应用软件在系统软件之上。



操作系统基础

- 操作系统：管理、控制和监督计算机软硬件资源的一系列程序的总称，是**最主要的系统软件**。
- 裸机：只有硬件系统。

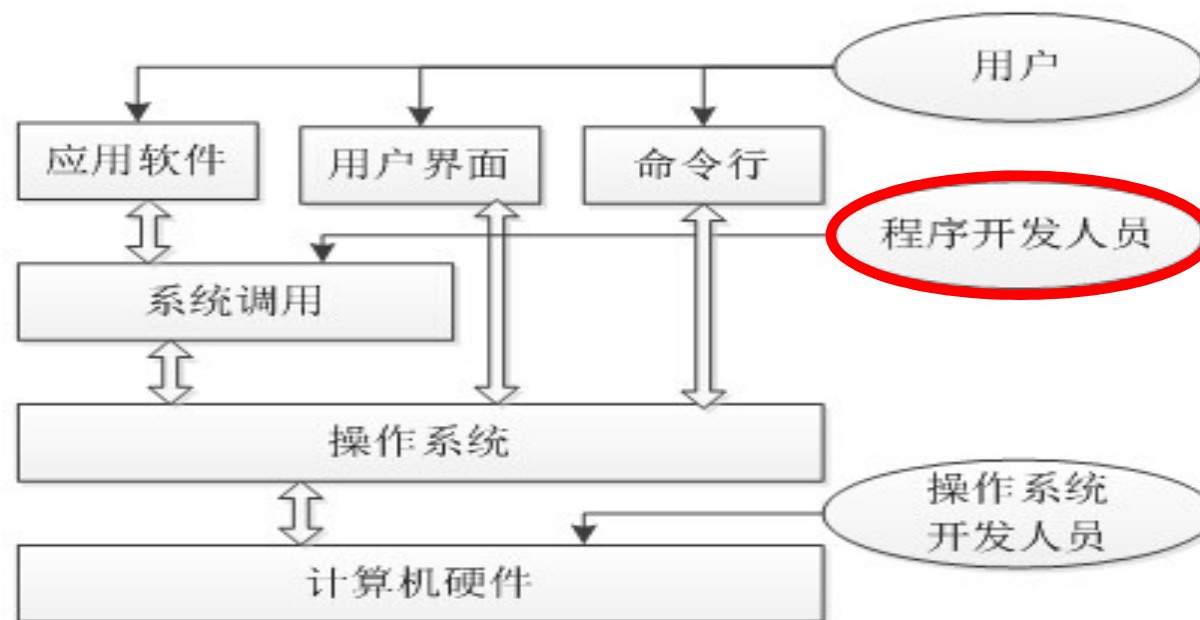
操作系统的主要特征

- ❖ 1. 并发性
- ❖ 2. 共享性
- ❖ 3. 异步性
- ❖ 4. 虚拟性



操作系统基础

人机交互与作业管理



操作系统基础

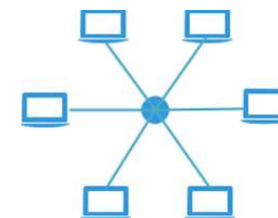
- 操作系统功能： **CPU管理； 存储管理； 设备管理； 信息管理**
- 单用户和多用户
- 单任务和多任务
- 常用计算机操作系统
 - MS-DOS： 单用户单任务， 字符界面
 - Windows： 单用户多任务， 图形界面
 - UNIX： 多用户多任务， 字符界面和图形界面
 - Linux： 多用户多任务， 字符界面和图形界面



网络基础

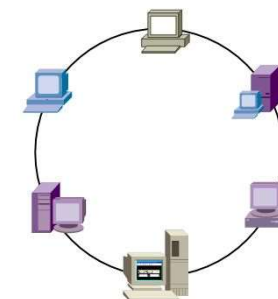
网络拓扑结构——星形结构

- 集中式控制：各个节点都与中心节点相连，中心节点控制全网的通信，任意两节点之间的通信必须经过中心节点。



网络拓扑结构——环形结构

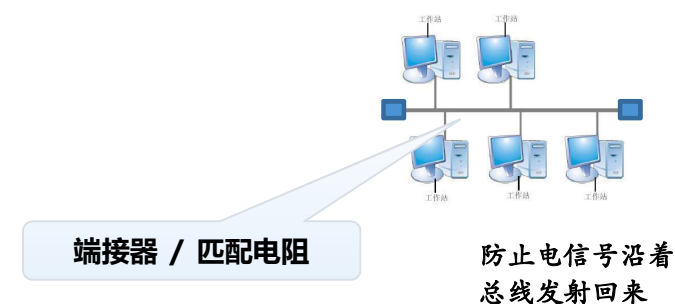
- 串行控制方式：所有节点连接成闭合环路，环中数据沿一个方向逐站传送。



网络基础

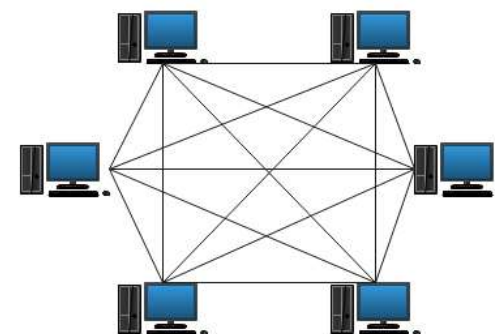
网络拓扑结构——总线结构

- 总线型结构：是将所有入网的计算机均接到一条通信线路上。网络中所有的站点**共享**一条数据通道（总线）。



网络拓扑结构——网状结构

- 网状结构是各结点通过传输线路互连起来形成网状，任何两个结点间可能有多条通路。



网络基础

互联网协议

■ TCP/IP

在Internet中包含异构网络，它们的硬件组成不同，运行的协议也不同。要将它们连接起来协调工作，就需要一个公认的协议：**传输控制协议和网际协议**
TCP/IP(transmission Control Protocol / Internet Protocol)是协议簇。

IP地址结构

IP地址采用分层结构，由**网络地址**和**主机地址**组成，地址长32位，分为4个字节。

网络基础

	1		2		3		4																																	
	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9										
A 类	0	网络标识符 (7 位)									主机编号 (24 位)																													
B 类	1	0	网络标识符 (14 位)														主机编号 (16 位)																							
C 类	1	1	0	网络标识符 (21 位)																			主机编号 (8 位)																	
D 类	1	1	1	0	多点广播地址 (28 位)																																			
E 类	1	1	1	1	0	实验保留地址																																		

A 类

126

1677214

1.0.0.1~126.255.255.254

B 类

16384

65534

128.0.0.1~191.255.255.254

C 类

20971152

254

192.0.0.1~223.255.255.254



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



➤ 图形与图像的区别

- **图形**是矢量概念，基本元素是图元；**图像**是位图概念，基本元素是像素；
- **图形**可以任意缩放无失真；**图像**缩放有失真；
- **图形**以图元为单位修改属性、编辑；**图像**只能对像素或图块处理；
- **图形**文件比较小，**图像**文件比较大
- **图形**适合描述轮廓不很复杂，色彩不是很丰富的对象，如几何图形，工程图纸等；**图像**适合表现含有大量细节（如明暗变化、场景复杂、轮廓色彩丰富）的对象，如：照片、绘图等。

数据处理及数据库

- **数据管理技术的发展：人工管理、文件管理、数据库管理**
 - 数据库DB；数据集合
 - 数据库管理系统DBMS：管理数据库的软件
 - 数据库系统DBS：带有数据库的计算机应用系统



➤ 数据模型:

概念模型: **E-R图**, 与**DBMS**无关

逻辑模型: 层次模型、网状模型和关系模型, 与**DBMS**有关

物理模型: 反映存储结构

➤ 关系模型: 二维表格, 一个关系是一张二维表。

➤ 表的每一行为一个记录 (元组)

➤ 表的每一列代表字段 (属性)

复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



语法要点--标识符

❖ 基本概念

- 字符集、标识符、关键字
- 各种常量的表示方法
- 变量的说明

标识符：C程序各种对象的名字。

标识符的组成：由字母或下划线开头的字母、数字或下划线序列。

abc, a12, _1aq a*3, 1aq, int ❌

关键字 在C语言中有特定含义的定义符。int, case, switch

scanf, define, include 等不是C语言关键字



语法要点--常量

● 整型常量的表示

10进制常量的表示 12

8进制常量的表示: 012 019 ✗

16进制常量的表示 0x12 0x2k ✗

● 实型常量的表示

小数形式: 1.2, 1.0, 1., 1.0, .1

指数形式: 2e3, 12e-2,

e2 ✗ 1.2e ✗ 3e1.2 ✗



语法要点--常量

- 字符型常量的表示—单引号

'A', '%', '8'

'AB' ✗

- 转义字符

'\n', '\t', '\r', '\b',

'\\', '\'', '\"'

*** (1到3位8进制数)

如: '\\101' '\\081' ✗, '\\1013' ✗

\\x** (1到2位16进制数)

如: '\\x41', '\\x2g' ✗



语法要点--常量

字符型变量

字符变量存储的是该字符的ASCII

关键字	字节	范围
char	1	

```
int main ( ) {  
    char c;  
    c = 'c';  
    printf("%c", c);  
    return 0;  
}
```

'a' ↔ 97

0 1 1 0 0 0 1



在一定范围内，整型数据和字符型数据可以混用。



语法要点--运算符

高 ↑	(), [], ., ->	
	~, !, ++, --, -, &, *, (类型), sizeof (右结合)	
	*, /, %	算术运算符
	+, -	
	<<, >>	移位运算符
	<, <=, >, >=	关系运算符
	==, !=	
	&	位运算符
	^	
	&&	逻辑运算符
	? :	(右结合)
	= (+=, -=, ...)	(右结合)
	,	

语法要点--运算符

- ◆ $i=1, j=3$ $++i + j-- + i++$ 的值是7 $i=3 \quad j=2$
- ◆ $x=3, y=4;$ $++x == y-- ? x++ : --y$ 的值是4
 $x=5 \quad y=3$
- ◆ $5/2$ 的值是2 $5/2.0$ 的值是2.5 (整数相除)
- ◆ $\text{printf}("%d", 5.4\%2);$ 出错 (求余, 整数)
 $\text{printf}("%d", (\text{int})5.4\%2);$ 的执行结果是 1
- ◆ $a=3, b=10$ $\text{if}(a=b) \quad b=a+5;$ $a=10 \quad b=15$ $(==)$
- ◆ $a=1, b=2, c=3$
 $a || c-- \ \&\& \ ++b$ 的值是1 $a=1 \quad b=2 \quad c=3$
 $!a \ \&\& \ (a = c--)$ 的值是0 $a=1 \quad c=3$
- ◆ $a=1, b=2, c=3$ $a+=b-=c*=3+a;$ $a=-9 \quad b=-10 \quad c=12$ (右结合)
- ◆ $a=1, b=2$ $\text{max}=a>b ? a : b$ $\text{max} = 2$
 $\text{min}=a>b ? b : (a<0.2 ? a : 0.2)$ $\text{min} = 0.2$



语法要点--运算符

◆ `k=(i=1, ++i, i++);` `k=2    i=3`

◆ `k=i=1, ++i, i++;` `k=1    i=3`

◆ `sizeof(int)` 的值是 4

若 `struct stu{`

`int num;`

`char name[19];`

`} a;`

`sizeof(a)` 的值是24, `sizeof(struct stu)` 的值是24



语法要点—数据类型转换

当不同数据类型的运算分量出现在同一个表达式中，C先将其转换为同一类型，然后运算。

1. 自动转换

- 允许不同的数据类型的值出现在算术表达式中。

- char 类型转换为int

- 单精度浮点类型float 转换为double类型。

- 整型数据和实型数据同时出现，结果为double类型。

double ←
↑
float
↑
long
↑
unsigned
↑
int ← char,
↑
short



语法要点—数据类型转换

思考：如果有 `int i; float f; double d; long e;`
则表达式 `10 + 'a' + i*f - d/e` 的结果是什么类型？

2. 类型转换运算符

功能：将表达式的结果强制转换为指定的类型。

格式：(类型) 表达式

`(int) (10 + 'a' + i*f - d/e)`

`(int)a` 将a的结果强制转换为整型量。

`(int)(x+y)` 将x+y的结果强制转换为整型量。

`(float)a+b` 将a的内容强制转换为浮点数，再与b相加



无论隐式转换，强制转换都是临时转换，不改变数据本身的类型和值。



3. 赋值转换

当赋值运算符两侧的类型不一致时，类型发生变化。

```
int x;  
float y = 5.3;  
x=y;  
X的值为5
```

将实型数据(无论单,双精度)
赋给整型变量时,舍弃小数.

```
int x=5;  
float y;  
y=x;  
Y的值为5.000000
```

整型数据赋给实型变量,数值
不变,但按实数形式存放。

语法要点——逗号运算

逗号运算符 用于连接几个表达式

(表达式 1, 表达式 2, 表达式 3, 表达式 n)

$X = ((a=4*5, a*2), \underline{a+6})$

逗号表达式的值为最后一个表达式的值

- 求解过程 顺序求解几个表达式
- 优先级 逗号运算符的优先级最低
- 结合性 左结合 $\longrightarrow$



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



流程控制

一个c程序可以有若干个源程序文件组成

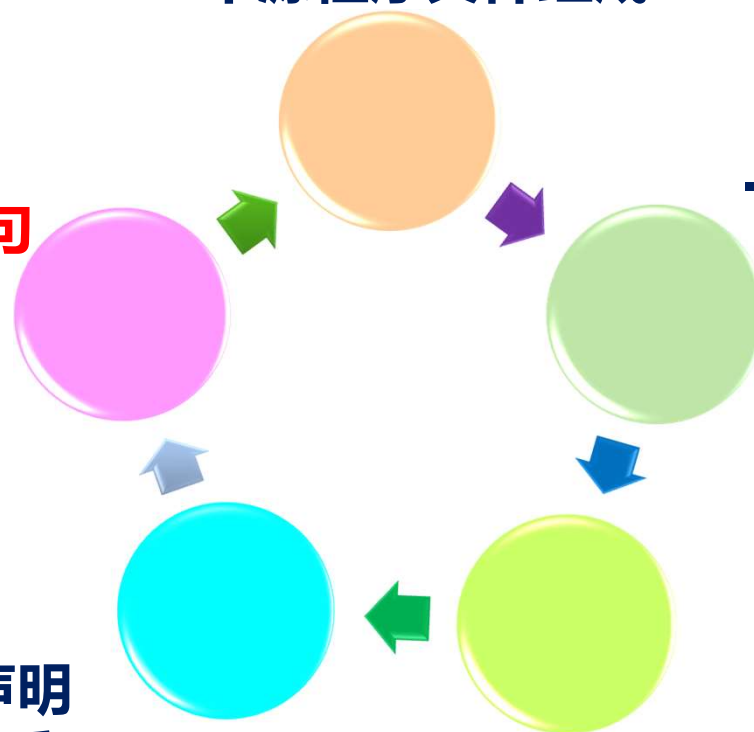
C语句分为

- 控制语句
- 函数调用语句
- 表达式语句
- 空语句
- 复合语句

一个源文件可以有若干个函数和预处理命令以及全局变量声明部分组成

函数体由数据声明
(说明语句) 和
执行语句组成

一个函数有函数首部
和函数体组成



流程控制

控制语句 完成一定的控制功能,实现三种基本结构

- | | | |
|------------------------------|---|-------------|
| 1) if() ~else 条件语句 | } | 选择结构 |
| 2) switch() 开关语句 | | |
| 3) for() ~ 循环语句 | } | 循环结构 |
| 4) while() ~循环语句 | | |
| 5) do ~while(); 循环语句 | | |
| 6) continue 继续语句 | | |
| 7) break 间断语句 | | |
| 8) goto 转向语句 | } | 函数调用 |
| 9) return 返回语句 | | |



流程控制

函数调用语句 由一个函数调用加一个分号构成一个语句
`printf( "Hello World.\n" );` **--库函数调用语句。**

表达式语句 表达式+分号
`a = 3 ;`

空语句 只有一个分号的语句 ;

- 用来做流程的转向点
- 用来作为循环语句中的循环体

复合语句 用一对{ }括起来的语句组，作为一条语句使用。
常用在循环结构和条件结构中。

```
int main()
{
    int a=1;
    printf("%d",a);
    {
        a=10;
        printf("%d", a);
    }
    printf("%d", a);
    return 0;
}
```



语法要点—赋值语句

1.赋值语句是由赋值表达式加上一个分号构成

例: `a=100` 赋值表达式
 `a=100;` 赋值语句

2.注意区分初始化与赋值语句

初始化: `int a=10;` 初始化语句出现在程序开始;
赋值语句: `int a;....a=10;` 赋值语句可以出现在程序任何位置。

3.条件中不能含有赋值符号, 但是赋值表达式可以包含于条件表达式中

例: `if(a=b)` `t=a;` 错误
 `if((a=b)>0)` `t=a;` 正确



语法要点--基本输入输出函数

- `ch=getchar( )`; 在键盘上读一个字符.

- 格式化输入函数

`scanf(格式串, 输入地址表)`

格式符: `d, ld, f, lf, e, le, c, s`

- ◆ 格式串中的普通字符要原样输入。

`scanf("a=%f", &a)` 输入: 12.3 ✗

`a=12.3` ✓

- ◆ 用`%lf(%ld)`输入`double(long)`变量的值

`double a; scanf("%lf", &a);`

- `%d`:以带符号的十进制形式输出整数
- `%o`:以八进制无符号形式输出整数
- `%x`:以十六进制无符号形式输出整数
- `%u`:以无符号十进制形式输出整数
- `%c`:以字符形式输出, 只输出一个字符
- `%s`:输出字符串
- `%f`:以小数形式输出单, 双精度数, 隐含输出六位小数
- `%e`:以指数形式输出实数
- `%g`:选用`%f`或`%e`格式中输出宽度较短的一种格式, 不输出无意义的0



语法要点--基本输入输出函数

常见的格式符的修饰符:

- 字母l: 用于长整型整数, 可加在格式符d, o, x, u前面
- 字母m (代表一个正整数): 数据最小宽度
- 字母n (代表一个正整数): 对实数, 表示输出n位小数; 对字符串, 表示截取的字符个数。
- —: 输出的数字或字符在域内向左靠对齐。



语法要点--基本输入输出函数

- ◆ 不能用%.nf的格式输入实型变量的值（可以指定宽度）

不能用scanf()输入带空格或跳格的字符串，为字符数组a输入：

How are you

scanf("%s", a); ✗

gets(a); ✓

scanf("%d, %4.1f", x, &y); ✗

scanf("%d, b=%f, %c%2d%c", &a, &b, &c, &d, &e);

输入：12, b=12.7, a123b<回车>

则：a=12 b=12.7 c='a' d=12 e='3'



语法要点--基本输入输出函数

- putchar(x) 将字符x输出到屏幕上。

```
char ch='#'; putchar(ch);  putchar('#');
```

- 格式化输出函数

```
printf(格式串, 输出表)
```

```
printf("%f, %e, %g\n", 123. 45, 123. 456, 123. 45);
```

输出 123. 450000, 1. 234560e+002, 123. 45



语法要点--基本输入输出函数

f 格式符。用来以小数形式输出实数（包括单双精度）

- **% f**。不指定字段宽度，**整数部分全部输出，并输出 6 位小数。**
- 在输出的数字中并非全部数字都是有效数字
- 单精度实数的有效位数一般为 7 位。

%m.n f。指定输出的数据共占m列，其中有 n 位小数。如果数值长度小于m，则左端补空格。

% - m.n f 与 **%m.n f** 基本相同，输出的数值左对齐右端补空格。



语法要点--基本输入输出函数

e 格式符，以指数形式输出实数

%e: 不指定输出数据所占的宽度和数字部分的小数位数.

例: `printf("%e", 123.456);`

输出:

<u>1 . 2 3 4 5 6 0</u>	<u>e + 0 0 2</u>
8列	5列

所输出的实数共占 13 列宽度。(注: 不同系统的规定略有不同)



语法要点--基本输入输出函数

`%m.ne`和`%-m.ne`。 `m`、`n`和“`-`”字符的含义与前相同。

此处`n`指拟输出的数据的小数部分（又称尾数）的小数位数。

若 `f = 1 2 3 . 4 5 6`，则：

```
printf("%e %10e %10.2e %.2e %-10.2e", f, f, f, f, f);
```

输出如下：

<u>1.234560e+002</u>	<u>1.234560e+002</u>	<u>1.23e+002</u>	<u>1.23e+002</u>
13列	13列	10列	9列
<u>1.23e+002</u>			
10列			

未指定 `n`，自动使 `n = 6`。

超过给定的 10 列，按实际长度输出。

第3个数据共占 10 列，小数部分占 2 列。

只指定 `n = 2`，未指定 `m`，自动使 `m` 等于数据应占的长度。

第5个数据应占 10 列，数值只有 9 列，由于是“`% - 10 . 2 e`”，数值向左靠，右补一个空格。

语法要点--基本输入输出函数

g 格式符，用来输出实数.

它根据数值的大小，自动选 f 格式或 e 格式（选择输出时占宽度较小的一种），且不输出无意义的零。

例:若 $f = 123.468$ ，则

```
printf ("%f %e %g", f, f, f);
```

输出如下:

<u>123.468000</u>	<u>1.234680e+002</u>	<u>123.468</u>
10列	13列	10列

说明:

- 用 % f 格式输出占 10 列，用 % e 格式输出占 13 列，用 % g 格式时，自动从上面两种格式中选择短者（今以 % f 格式为短）
- 故占 10 列，并按 % f 格式用小数形式输出，最后3个小数位为无意义的 0，不输出，因此输出 123.468，然后右补3个空格。

流程控制

基本语句——必须会

掌握正确的语法形式、控制流程、常见书写形式

if 和 switch 语句

循环语句 (for、while、do-while)

break 和 continue 语句

语句之间的嵌套关系和规定

最基本的多重循环程序的编写方法



语法要点--if语句

- if语句

- ◆ if(p) s

- ◆ if(p) s1
 else s2

- ◆ if(p1) s1
 else if(p2) s2
 else if(p3) s3
 else s4

p, p1, p2, p3是任何表达式,
非0为真, 0为假.

- 匹配规则：在嵌套的if-else语句中，else子句总是与上面的、离它最近的、且在同一复合语句中还没有配对的if子语配对。



语法要点--if语句

◆ `x=0;`

`if(x = 0) s1; s2; s3;`

执行语句: s2、s3

◆ `x=0;`

`if(x == 0) s1; s2; s3;`

执行语句: s1、s2、s3

◆ `x=0;`

`if(x = 1) s1;s2;s3;`

执行语句: s1、s2、s3

◆ `if(n) s1;`

其中 n 等价于: `n != 0`

`if(!n) s1;`

其中!n等价于: `n == 0`



语法要点--开关语句

- switch语句

```
switch(K)
{ case K1:  S1
  case K2:  S2
    .....
  case Kn:  Sn
  default: Sn+1;
}
```

K 整型或字符型表达式

K_i 整型或字符型常量表达式

S_i 语句



语法要点--开关语句

switch后面括弧内的“表达式”，一般使用整型或字符型

多个可以共用一组执行语句

说明

每一个case的常量表达式的值必须互不相同

执行一个case后面的语句后，流程控制转移到下一个case继续执行。在执行一个case分支后,可以用一个break语句来终止switch语句的执行。

各个case和default的出现次序不影响执行结果。



语法要点--开关语句

```
switch(k);          /* 不能出现分号 */
{
    case1:      j=1;    /* case 和1应分开: case 1: */
    case 1.2:    j++;    /* case后为整型或字符型常量 */
    case 1+1:    k++;
    case 2:      h--;    /* 不能出现相同case */
    case a:      g++;    /* 不能出现变量 */
}
```

```
int i=10, j=0;
switch(j=i) {
    case 0:  i+=1;    /*(1)*/
    case 10: i/=2;    /*(2)*/
    default: i%=3;
}
```

i=2 若在(1), (2)后加break; , 则i=5

语法要点--开关语句

❖ Break语句格式

break;

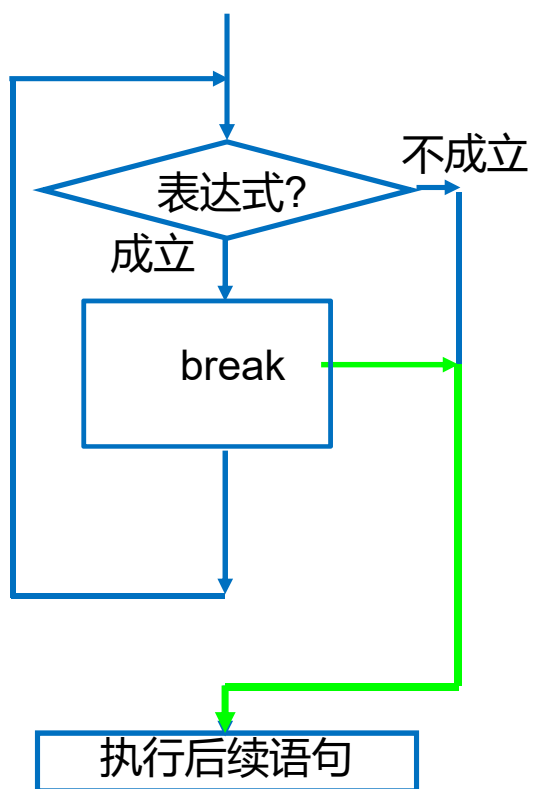
❖ break语句的功能

- 1.在**switch**语句中结束**case**子句，使控制转到**switch**语句之外。
- 2.在**循环**语句的循环体中使用，结束循环过程，使控制转移到整个**循环**语句之外的下一条语句处。

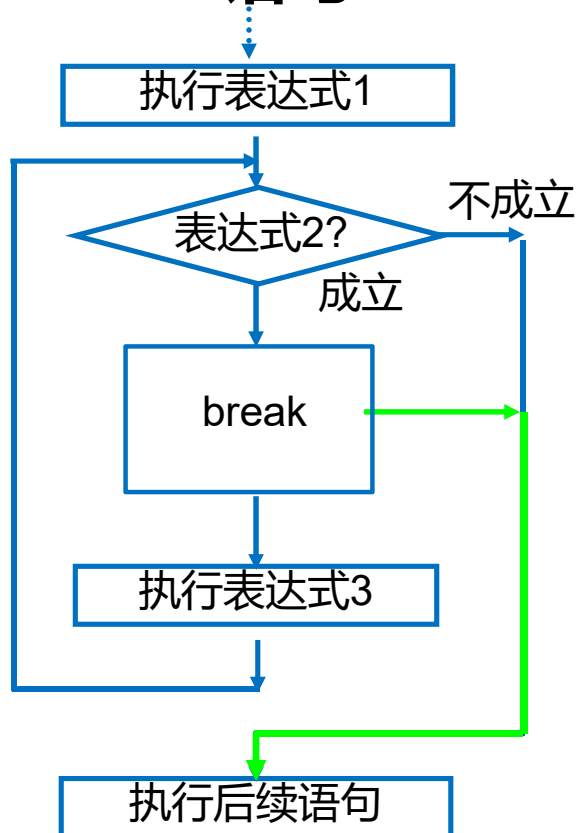


语法要点--循环语句

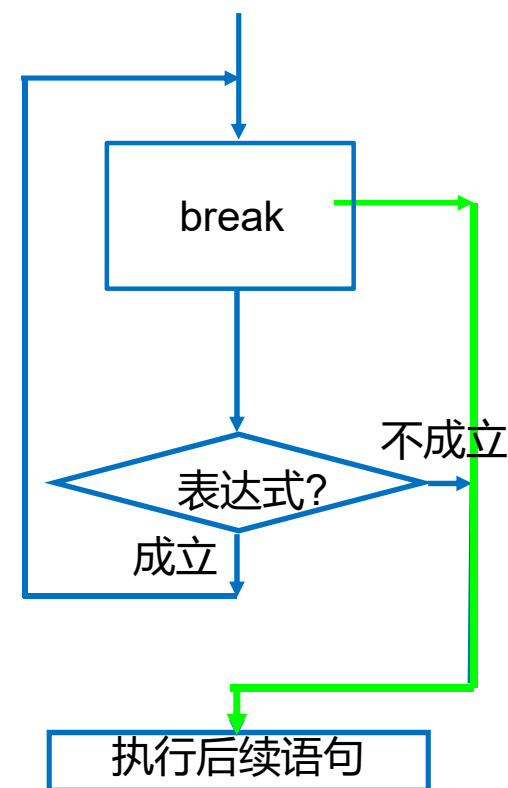
while语句



for语句



do-while语句



语法要点--循环语句

❖ break语句使用注意:

- 1、在嵌套循环中，break 语句仅能退出一层(当前层)循环。
- 2、若在循环语句中包含了 switch 语句，那么switch 语句中的 break 语句仅能使控制退出 switch 语句。
- 3、break 语句并不是程序设计中必不可少的语句，可以通过改变程序的结构去掉。



语法要点--循环语句

● 3种循环语句

`while (p) {...循环体...}`

`do {...循环体...} while (p) ;`

`for(k1;k2;k3) {...循环体...}`

`i=0; while(i++<3) ;` 循环3次, 循环后*i*=4

`i=0; while(++i <3) ;` 循环2次, 循环后*i*=3

`i=0; do { s+=i; } while(i++<3);`

`i=0; do s+=i; while(i++<3);` 循环4次

`for(j=0;j<6;j++) s+=j;`

`for(j=0;k=0;j<6 && k!=4;k++, j++) s+=j;` 循环4次

`for(j=0; ;j++) s+=j;` 死循环



语法要点--循环语句

● continue与 break语句区别

◆ continue只能出现在循环语句中

break 出现在循环语句或switch语句中

◆ continue 结束本次循环, 开始下次循环的判断

break 结束整个循环

```
for(i=0; i<8; i++)  
{ if(i < 5) continue; /* break; */  
printf("%5d", i);}
```

输出 5 6 7

若continue; 换成break; 则输出什么? 无输出



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



语法要点——数组

数组——重点

- 数组的概念，数组说明，数组元素起始下标，如何引用数组元素，数组名的意义
- 多维数组元素的标识方法，存放方式。多维数组只要求到二维
- 数组与函数的关系：在函数之间传递数组中的一个元素，在函数之间传递整个数组
- 基本算法：排序（冒泡、选择），求最大(小)值, 矩阵乘法，矩阵转置



语法要点——数组

● 数组定义、函数传递数组 / 字符串数组（区别）

```
#define N 2
```

```
int a[3], b[N][2];          int c[k]; ❌ a[3]=23; ❌
```

数组名代表数组首地址: $a \equiv \&a[0]$

```
static int a[5]={1, 2, 3, 4, 5},
```

```
        b[2][3]={0, 1, 2, 10, 11, 12},
```

```
        c[ ]={1, 2},    d[ ][2]={1, 2, 3};
```

```
char d[ ]="china" ;
```

```
int a[4]={1, 2, 3, 4, 5} ❌ 初值个数不能多于数组元素个数
```

```
char a[5]="12345"; ❌ 字符串共有6个字符, 即china\0
```

```
a={1, 2, 3, 4, 5} ❌      数组名是地址常量
```



语法要点——数组

三种排序算法

Bubble Sort (冒泡排序)

Selection Sort (选择排序)

Insertion Sort (插入排序)



语法要点——字符串

●字符串——重点

◆字符串常量：用双引号括起的一串字符，系统自动在串尾加‘\0’（串结束标记）。 “china”， ”1”

◆用字符数组存放字符串

```
char a[10]="china", b[4]="1234" ×
```

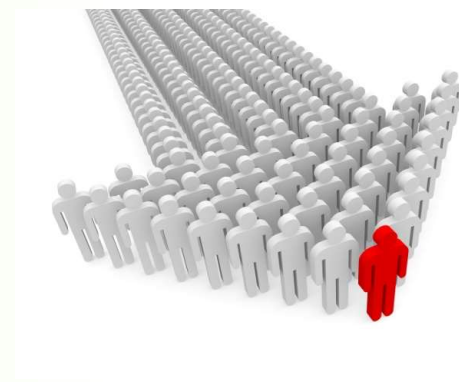
◆串输入/输出函数

gets(s) 从键盘上输入字符串并存入s数组

puts(s) 输出s串，并换行

◆串的基本操作

复制、连接、求长度、比较、反向、查找、合并、排序、求子串。



语法要点——字符串

★ 字符串反向

```
void revstr(char a[])
{
    char c;    int s=0; p= strlen(a) -1;
    while(s<p) {        /* 对称位置字符交换位置 */
        c=a[s]; a[s]=a[p];    a[p]=c;
        s++;  p--;        /* 下标s加1, 下标p减1 */
    }
}

int main( )
{
    char str[80] ;
    gets(str);    /* 输入字符串到str数组 */
    revstr(str); /* 调用 函数实现字符串反向 */
    puts(str);    /* 输出反向后的字符串*/
    return 0;
}
```



语法要点——字符串

```
void revstr(char *p)    /* 指针法实现字符串反向 */
{
    char c, *q = p + strlen(p) - 1;
    while(p < q) {      /* 对称位置字符交换位置 */
        c = *p; *p = *q; *q = c;
        p++; q--;       /* p加1, q减1 */
    }
}

int main( ) {
    char str[80] ;
    gets(str);           /* 输入字符串到str数组 */
    revstr(str);         /* 调用 函数实现字符串反向 */
    puts(str);           /* 输出反向后的字符串 */
    return 0;
}
```



语法要点——字符串

★两个升序字符串合并

```
include <stdio.h>

int main( ) {
    char s1[80], s2[80], s0[80];  int i, j, n;
    gets(s1);      gets(s2);
    for(i=0, j=0, n=0; s1[i] != '\0' && s2[j] != '\0'; )    /*串合并 */
        if(s1[i] < s2[j]) s0[n++] = s1[i++];    /*将较小字符存入s0 */
        else
            s0[n++] = s2[j++];
    while(s1[i] != '\0')  s0[n++] = s1[i++]; /*继续处理s1剩余字符 */
    while(s2[j] != '\0')  s0[n++] = s2[j++]; /*继续处理s2剩余字符 */
    s0[n] = '\0';    /* 最后补上结束符 */
    puts(s0);
    return 0;
}
```



语法要点——字符串

★在多个字符串中查找字符串

```
include <stdio.h>

int main( ) {
    char *a[4]={“BASIC”, “ALGOL”, “CPL”, “C”}, *p, s[10];    int i=0;
    gets(s);
    while(i < 4) {
        if(strcpm(s, a[i])==0) {
            printf(“%s=a[%d]=%s\n”, s, i, a[i]);
            break;
        }    else    i++;
    }
    if(i == 4)    printf(" %s not found!\n ", s);
}
```



语法要点——字符串

★求字符串中子串

```
include <stdio.h>

main( ) {    /*求字符串从位置b开始的n个字符组成的子串*/
    char a[80] = "BASICALGOLCPLC ", s[20];  int b, n, i, j=0;
    scanf(" %d, %d ", &b, &n);
    for(i=b; i<b+n; i++)  s[j++] = a[i];
    s[j] = '\0';
    printf(" %s , %s\n ", a, s);
}
```



字符类数据要点总结

- 1 字符变量是一个字符;
- 2 字符常量中有转义字符;
- 3 字符串结尾有' \0' , 占1个字节;
- 4 字符串输入时用scanf, %s遇空格结束;
- 5 字符数组输入用gets可以有空格;
- 6 字符数据存ASCII;
- 7 字符数组char s[10]= “china” , 正确;
- 8 字符指针 char *s, s= “china” , 正确;
- 9 字符串处理函数中的参数有的可以用字符串常量;
- 10 字符串处理函数中的参数的含义是地址;
- 11 字符串不可以赋值, 必须用strcpy
- 12 字符串不可以直接判断恒等, 必须用strcmp
- 13 字符指针与字符数组区别



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



语法要点--函数

● 定义

```
float f(int x)    /*函数头*/  
{ int y;         /* 函数体 */  
  y=x*x;  
  return y;  
}
```

不能嵌套定义



● 调用函数

```
float max(int x, int y) /* x, y是形式参数 */  
{.....}  
c = max(a, b);          /* a, b是实际参数 */
```

● 实际参数和形式参数: 个数相等, 类型一致



语法要点--函数

- 函数返回值的类型取决于函数类型

```
int f( ) {          /* f( ) */  
    float x=2.3;  
    return x+1.3; /* 返回3 */  
}
```

- ◆ float* f()
 {.....} /* 返回float指针*/

- ◆ void f()
 {.....} /* 无返回值*/

- ◆ struct node f()
 {.....} /* 返回struct node 值 */



语法要点——函数

● 参数的传递

实参	形参	传递的值
常量, 变量, 表达式	变量	值 (单向值传递)
数组	数组, 指针变量	数组的首地址
变量地址	指针变量	地址
结构地址	结构指针变量	结构地址



语法要点——函数

●递归调用

按递归公式写程序:

$n! = f(n) = 1$	$n = 1$
$n * f(n-1)$	$n > 1$

```
long f(int n ) /*求n! */
{
    if(n==1) return 1;
    else return n*f(n-1);
}
```

```
int main( )
{
    printf("%ld", f(n));
    return 0;
}
```



语法要点——函数

★递归法求字符串长度

```
/* strlen1(s+1)是以(s+1)为首地址的串长*/  
int strlen1 (char *s) /* 求s为首地址的串长 */  
{  
    if(*s=='\0')    return  0;  
    else            return  1+strlen1(s+1);  
}  
  
int main( ) {  
    char s[80];  
    scanf ("%s", s);  
    printf ("%d\n", strlen1(s));  
    return 0;  
}
```



语法要点一函数

★递归法反向输出整数各位数字

```
void f (int n)
{
    if(n>=0 && n<=9)  printf("%d", n);  /*一位数字直接输出 */
    else {
        printf("%d", n % 10);      /* 先输出个位数字 */
        f(n/10);                  /* 输出个位以前的各位数字*/
    }
}

int main( ) {
    int n = 12345;
    printf("%ld",f(n));
    return 0;
}
```



语法要点--函数

存储类型小结

性 能	自动变量	外部变量	外部静态	内部静态	寄存器变量
记忆能力	无	有	有	有	无
多个函数共享	否	可	可	否	否
在不同文件共享性	否	可	否	否	否
未显示赋值的取值	不定	0	0	0	不定
变量初始化	程序控制	编译器	编译器	编译器	程序控制
数组与结构初始化	有	可	可	可	否
作用域	当前函数	整个程序	文件	函数	当前函数



函数要点总结

- 1 函数是C中最小的功能单位；
- 2 程序可以有多个文件，一个文件可以有多个函数组成；
- 3 一个程序的运行必须从main开始；
- 4 函数的类型是返回值的类型，不定义就是int
- 5 函数定义类型与返回值类型不符，以定义为准；
- 6 函数形参是局部变量，仅在函数调用时有存储空间有值
- 7 若实参与形参都是基本变量，调用时是单向值传递
- 8 函数可以嵌套调用；
- 9 递归函数必须有结束的条件；常用的几个递归函数。
- 10 数组名做函数参数传递的是地址
- 11 形参是指针，实参可以是变量的地址
- 12 函数可以有多个返回语句，也可以没有返回语句；
- 13 void是无类型函数，无返回值；
- 14 调用函数前必须对其做说明
- 15 外部变量，静态变量占内存静态部分，不赋初值时为0；
- 16 不定义存储类型的系统默认为auto型
- 17 编译预处理不是语句，是特殊命令
- 18 函数调用可以是语句，也可以是表达式的运算分量
- 19 正确判断函数的实参或形参（变量？指针？结构变量？数组？）



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



语法要点 一 指针

指针——重点

- 指针变量的基本概念与基本运算：取地址 “&” 运算，取内容 “*” 运算
- 指针变量与一般变量的关系
- 指针与函数的关系
- 数组与指针的关系，指针的3种基本运算与意义
 - 指针与整数进行加减运算（指针变量与自增++、自减--运算的关系）
 - 指针之间进行关系运算
 - 指针之间求差值
- 指针数组与数组指针，main函数的参数



语法要点 一 指针

- 指针：指针变量专门存放地址。
- ◆ 存储数据：指向变量的地址（指针）。

`int a, *p;` `p=&a;` $*p \Leftrightarrow a$

- ◆ 指向数组元素的指针

`int a[10], *p=a;`

$p+i \Leftrightarrow \&a[i] \Leftrightarrow a+i$

$p[i] \Leftrightarrow *(p+i) \Leftrightarrow a[i] \Leftrightarrow *(a+i)$

如： $a[i] \Leftrightarrow a[j] \Rightarrow$ `t=*(a+i); *(a+i)= *(a+j); *(a+j)=t;`
 `t=a[i];` `a[i]=a[j];` `a[j]=t;`



语法要点 一 指针

◆ 指向二维数组的指针

`int a[3][3], *p;` 指针变量p的基类型是int

`p=a[0]; (p = &a[0][0];)`

`p+i*n+j` $\Leftrightarrow$ `&a[i][j]` (n是列数)

`*(p+i*n+j)` $\Leftrightarrow$ `a[i][j]`

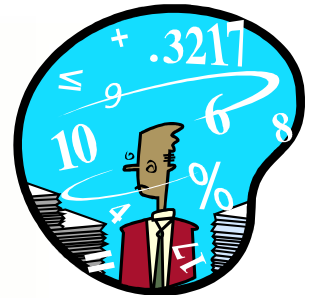
◆ 指向一维数组的指针

`int a[3][3], (*p)[3];` 指针变量p的基类型是int [3]

`p=a;`

`*(p+i)+j` $\Leftrightarrow$ `&a[i][j]` $\Leftrightarrow$ `*(a+i)+j` $\Leftrightarrow$ `a[i]+j`

`*(*(p+i)+j)` $\Leftrightarrow$ `a[i][j]`



语法要点 一 指针

◆ 字符指针

`char s[ ] = "china", *p;` 指针变量p的基类型是char

`p = s;`

`p+i ⇔ &s[i]            *(p+i) ⇔ s[i]`

`printf("%c,%s\n", *(s+2), s+2);` 输出i, ina

★ 字符数组与字符指针的异同

同: `char s[] = "china", *p = "123";`

`printf ("%s,%s\n", s, p);`

异: `p = "abc";` p 是指针变量

`s = "xyz";` ✗ s 是地址常量



语法要点 一 指针

◆ 指针的3种基本运算与意义

1. 指针与整数进行加减运算

```
int a[10], *p=&a[3];
```

```
p+1  $\Leftrightarrow$  &a[4]      p-1  $\Leftrightarrow$  &a[2]
```

```
++p  $\rightarrow$  p=p+1  $\rightarrow$  p: &a[4]
```

```
*p++  $\rightarrow$  *(p++)  $\rightarrow$  *p 和 p=p+1
```

```
p=&a[2]; printf("%d\n", *p++);      输出a[2]
```

```
printf("%d\n", *p);                输出a[3]
```

```
*++p  $\rightarrow$  *(++p)  $\rightarrow$  p=p+1 和 *p
```

```
p = &a[2]; printf("%d\n", *++p);    输出 a[3]
```

```
printf("%d\n", *p);                输出 a[3]
```



语法要点 一 指针

2. 指针之间进行关系运算

两个指针指向同一数组的元素时，才能进行指针关系运算。

```
int a[10], p = &a[2]; q = &a[4];
```

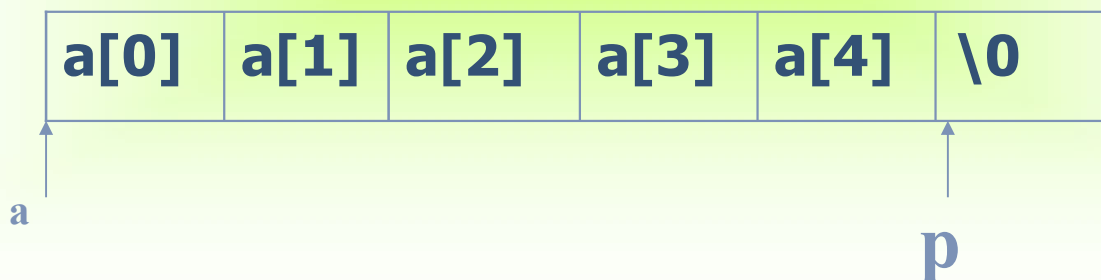
```
printf("%d", p < q); 输出 1
```

```
for(p = a, q = a + 9; p <= q; p++)
```

```
    printf("%d\n", *p);
```



3. 指针减法运算



p - a 的值为5，是p, a之间元素的个数，即“china”的长度。



语法要点 一 指针

◆ 函数指针

★ 函数指针： 函数的入口地址 (用函数名表示)

```
int max(int a, int b);  
int (*p)(int, int);
```

★ 通过函数指针调用函数

```
p = max;  
t = (*p)(a, b);    ⇔    t = max(a, b);
```



语法要点 一 指针

◆ 指针数组

```
char *a[3]={"china", "beijing", "nanjing"};
```

a[i] 存放第i个字符串的首地址。

如：a[1] 存放 “beijing” 首地址。

用途：处理多个字符串(如:排序、求最大(小)串等)

```
int main( ) {  
    char *a[3] = {"china", "beijing", "nanjing"};  
    int k;  
    for(k=0; k<3; k++)    printf("%s**", a[k]);  
}
```

输出： china**beijing**nanjing**

指针要点总结

- 1 指针就是地址，指向不同目标，定义形式不同；
- 2 指针必须用地址类数据初始化（数组名，&变量名，其他指针）
- 3 指针指向变量，可以通过指针间接访问变量；
- 4 指针指向一维数组，可以访问数组中所有元素，要点是 $p+1$ 指指针向后移动一个元素
- 5 两个指针指向同一数组时，可以有关系运算，部分算术运算
- 6 定义`int a[10]`后，`a`，`a+1`，`a+l`，`*(a+i)`，`a[i]`的含义
- 7 数组名是常量，不能做运算，`int a[10]`，`a++`，`a=p+3`等是错的。
- 8 字符数组`char s[10]="china"`，正确；
- 9 字符指针 `char *s`，`s="china"`，指针`s`仅指向字符串首；
- 10 字符串处理函数中的参数可以是指针名；
- 11 指针数组必须用地址值初始化；
- 12 行指针必须指向二维数组中的行，该指针加整数含义是指到后续的行（不是元素）
- 13 二级指针，指针数组，二维数组是等价的（各种表达式）
- 14 二维数组定义后 `int a[3][4]`，正确判断如何代表地址，如何求元素的值
- 15 指向函数的指针与返回指针的函数。看懂ppt中的程序。
- 16 指针指向结构体变量，引用结构体成员有两种形式。
- 17 指针可以赋空值NULL



复习内容

1	计算机概述及数字化基础
2	计算机三大平台——硬件、软件、网络
3	数据处理及数据库
4	数据类型，运算符，表达式
5	流程控制——顺序、选择、循环
6	数组
7	函数
8	指针
9	结构



语法要点 一结构

结构

定义结构类型、结构变量，结构占用的内存

与结构变量有关的运算符：访问结构成员的方法

结构与函数的关系：函数间传递结构

结构与数组的关系，结构数组

结构与指针的关系，结构指针与++、--运算的关系

联合

◆ 定义联合类型、联合变量，联合占用的内存

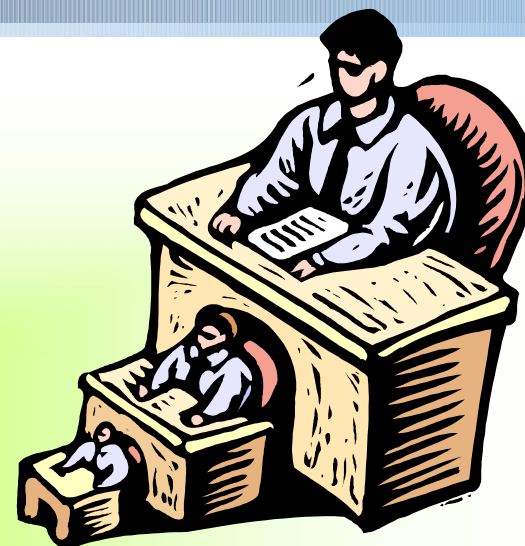
◆ 访问联合成员的方法



语法要点 一结构

● 结构定义

```
struct stu  
{ int num;  
  char name[10];  
}  
a={1001, "li"}, b[5], *p=&a;  
struct stu d, e[5], *q=e;
```



● 结构指针

`p = &a.num;` ✗ `p` 是结构型指针, 而 `a.num` 是 `int` 型。



语法要点 一结构

● 引用结构成员

引用结构成员的3种方法：

结构变量. 成员	如： a. num
(*结构指针). 成员	如： (*p). num
结构指针->成员	如： p->num

● 函数间传递结构数据方法：

1. 参数用结构指针(地址)
2. 参数用结构变量
3. 参数用结构变量各成员

其中, 以传递结构指针最方便



语法要点 一结构

输入书的书名和单价，并输出

```
struct book {  
    char name[20];    /*书名 */  
    float price;      /*单价 */  
};  
  
void printbook(struct book *p)  
{ printf("%-20s %6.2f\n", p->name, p->price); }  
  
int main( ) {  
    struct book books[10]; int k;  
    for(k=0; k<10; k++)    /*输入书名, 单价 */  
        scanf("%s%f", books[k].name, &books[k].price);  
    for(k=0; k<10; k++)    /*输出10本书的书名, 单价 */  
        printbook(&books[k]); /*输出第 k 本书书名、单价 */  
}
```



语法要点 一结构

```
struct data {  
    char name[20];  
    int c;  
} a[3] = {"wang", 10, "fun", 2, "li", 4}, *j, *p=a;
```

$a+i \Leftrightarrow \&a[i]$ $p+i \Leftrightarrow \&a[i]$

$p++ \rightarrow c; \Rightarrow p \rightarrow c, p=p+1$

$(p++) \rightarrow c; \Rightarrow p \rightarrow c, p=p+1$

$++p \rightarrow c; \Rightarrow ++(p \rightarrow c) \Rightarrow p \rightarrow c+=1$

$(++p) \rightarrow c; \Rightarrow p=p+1, p \rightarrow c$

$p \rightarrow c++; \Rightarrow (p \rightarrow c)++ \Rightarrow p \rightarrow c+=1$



语法要点 一结构

```
struct s { int x, *y; };
int *p, data[5] = {10, 20, 30, 40, 50};
struct s array[5] = { 100, &data[0], 200, &data[1], 300, &data[2],
                    400, &data[3], 500, &data[4] };

int main( ) {
    p = array;
    printf("%d", p->x)      /*输出      100  */
    printf("%d", (*p).x)   /*输出      100  */
    printf("%d", *p->y)     /*输出 *&data[0]: 10 */
    printf("%d", *(*p).y)  /*输出      10  */
    printf("%d", ++p->x)    /*输出      101  */
    printf("%d", (++p)->x) /*输出 p=&array[1]: 200*/
    printf("%d", p->x++)    /*输出      200 p->x =201 */
    return 0;
}
```

结构体要点总结

1. 区分结构体类型、结构体成员和结构体变量；
2. 结构体变量三种定义形式；
3. 结构体变量初始化，注意不同类型的常量；
4. 嵌套的结构体变量成员如何引用；
5. 结构体数组和结构体指针引用结构体成员时的自增、自减的含义；
6. 结构体变量所占存储空间的计算，sizeof的使用；
7. 结构体整体做函数实参和形参；
8. 如何用结构体指针引用结构体成员。



编程题目总结

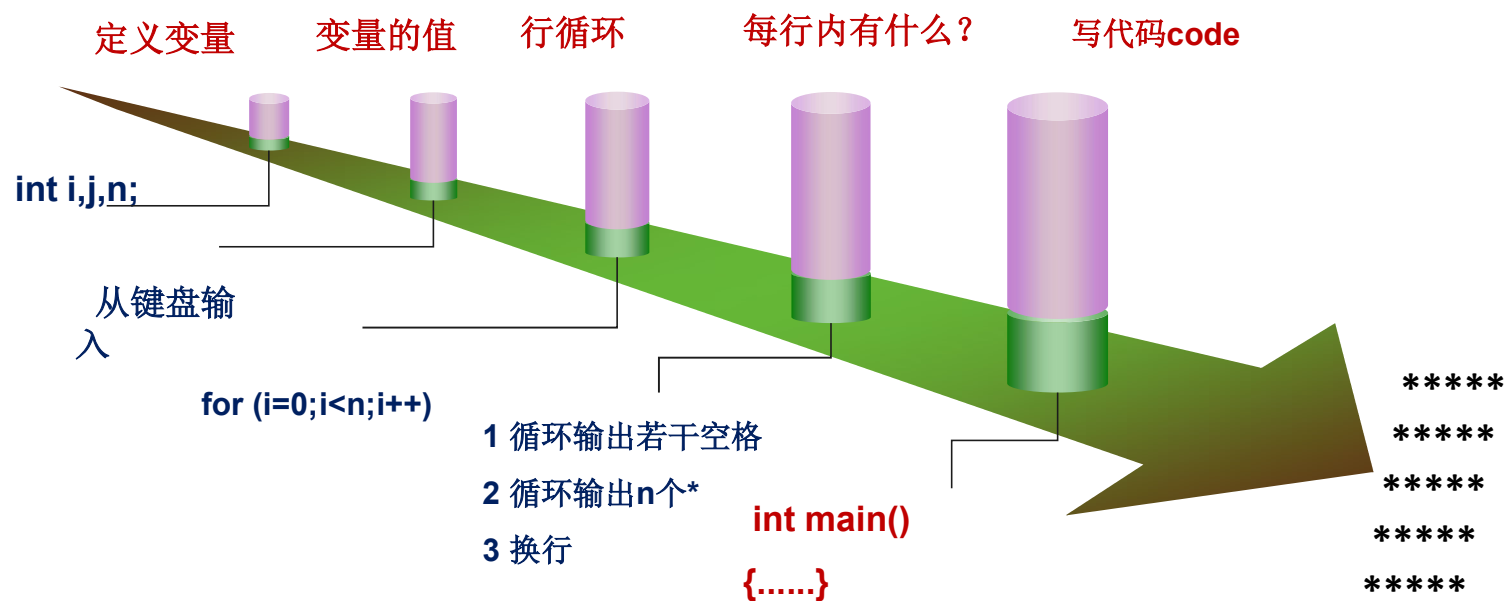
- 输出图形 (*, n 行 & n 列, n 行 & $2*n-1$ 列, $2*n-1$ 行 & n 列, $2*n-1$ 行 & $2*n-1$ 列, 或其他三角形, 四边形, 梯形, 字母形状等, 图形的组成是数字 1--9, 或是字母 a--z, A--Z) ;
- 数值类: 与日期相关的, 与素数相关的, 与数值转换相关的, 与数组相关, 与排序相关, 与最大值最小值相关;
- 字符串操作: 一个字符串的查找, 排序, 插入, 删除, 多个字符串的合并, 排序, 查找, 插入, 删除等。



编程题目总结

题目：键盘输入n，输出平行四边形，例如输入5

步骤：键盘输入n，输出n行，每行先有若干个空格，然后n个*



编程题目总结

```
1 #include<stdio.h>
2 int main( )
3 { int i,j,n;
4   scanf("%d",&n);
5   for(i=0;i<n;i++)
6   { for(j=0;j<n-i;j++)
7     printf(" ");
8     for(j=0;j<n;j++)
9       printf("*");
10    printf("\n");
11  }
12  return 0;
13 }
```

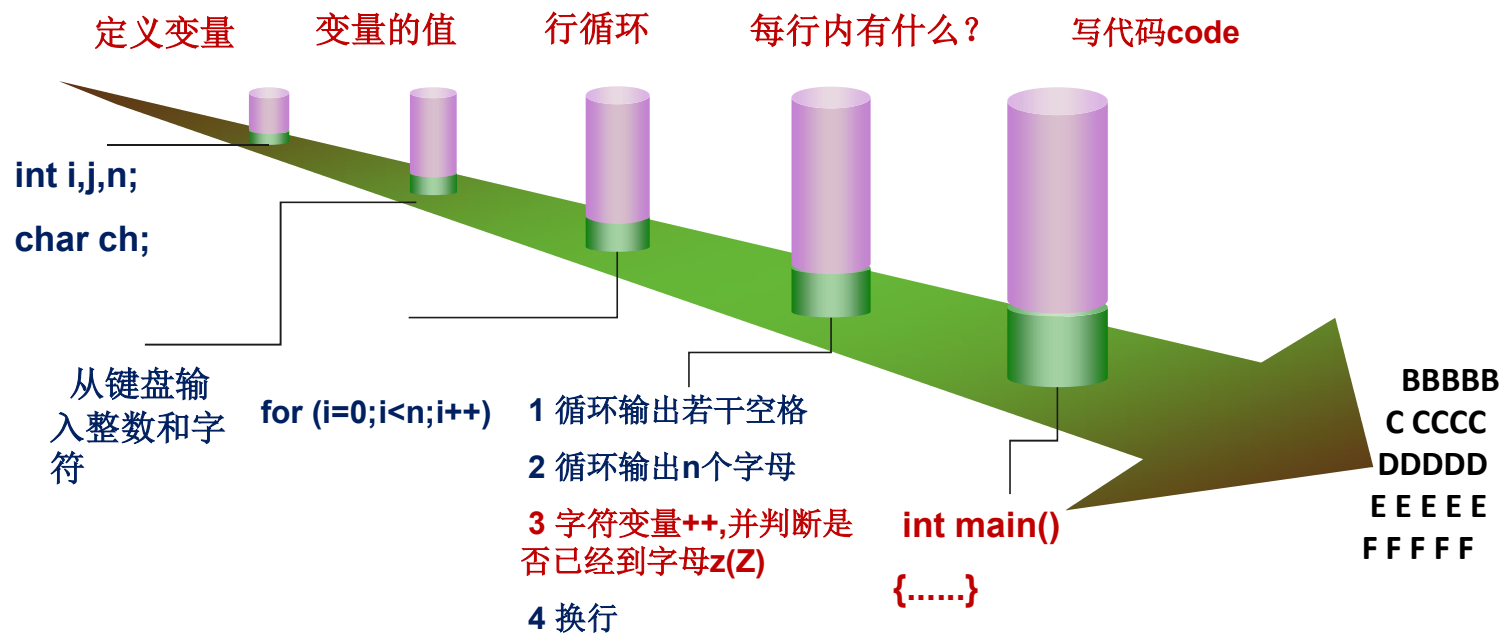
```
5
    *****
    *****
    *****
    *****
    *****
-----
Process exited with return value 0
Press any key to continue . . .
```



编程题目总结

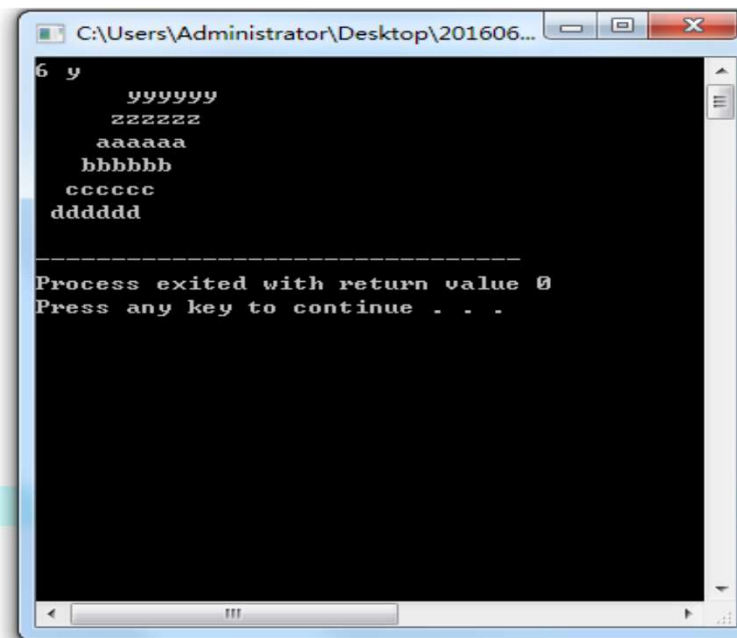
题目：键盘输入n和一个字母，输出字母组成的平行四边形，例如输入5， B

步骤：键盘输入n，输出n行，每行先有若干个空格，然后n个字母，字母变化的规律是



编程题目总结

```
1  #include<stdio.h>
2  int main( )
3  {   int i,j,n;
4      char ch;
5      scanf("%d %c",&n,&ch);
6      for(i=0;i<n;i++)
7      {   for(j=0;j<n-i;j++)
8          printf(" ");
9          for(j=0;j<n;j++)
10             printf("%c",ch);
11             if(ch=='z' || ch=='Z')
12                 ch=ch-26;
13                 ch++;
14             printf("\n");
15         }
16     return 0;
17 }
```



```
C:\Users\Administrator\Desktop\201606...
6 y
      yyyyyyy
      zzzzzzz
      aaaaaaa
      bbbbbb
      cccccc
      ddddd

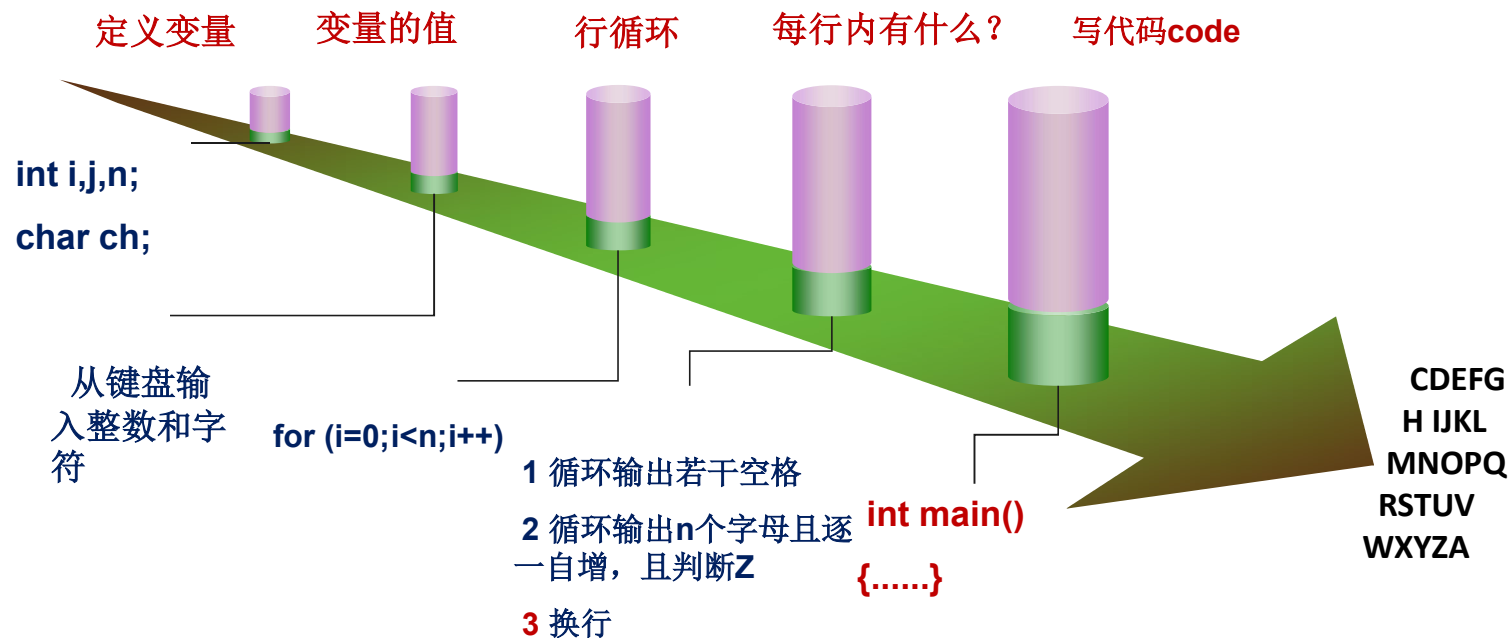
-----
Process exited with return value 0
Press any key to continue . . .
```



编程题目总结

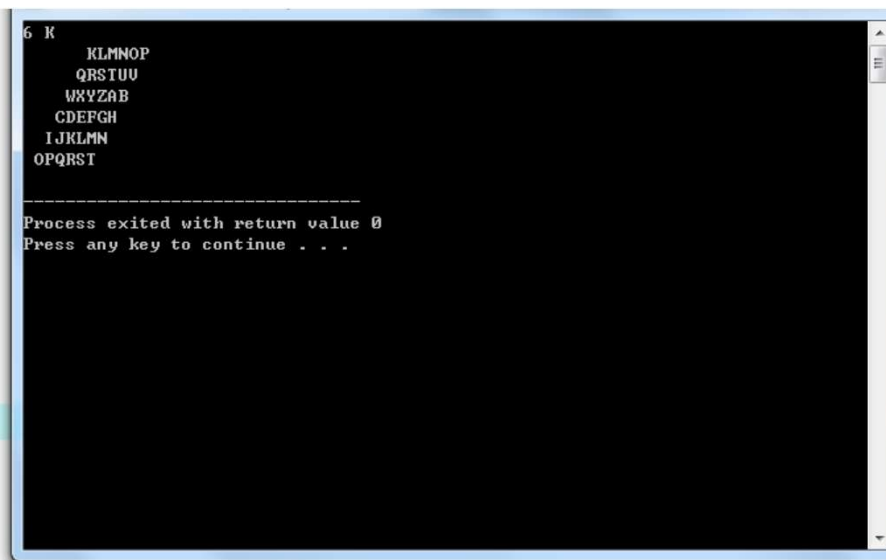
题目：键盘输入n和一个字母，输出字母组成的平行四边形，例如输入5，C

步骤：键盘输入n，输出n行，每行先有若干个空格，然后n个字母，字母变化的规律是



编程题目总结

```
1  #include<stdio.h>
2  int main( )
3  {   int i,j,n;
4      char ch;
5      scanf("%d %c",&n,&ch);
6      for(i=0;i<n;i++)
7      {   for(j=0;j<n-i;j++)
8          printf(" ");
9          for(j=0;j<n;j++)
10         {   printf("%c",ch);
11             if(ch=='z' || ch=='Z')
12                 ch=ch-26;
13             ch++;}
14         printf("\n");
15     }
16     return 0;
17 }
```



```
6 K
  KLMNOP
 QRSUU
WXYZAB
 CDEFGH
  IJKLMN
   OPQRST

-----
Process exited with return value 0
Press any key to continue . . .
```

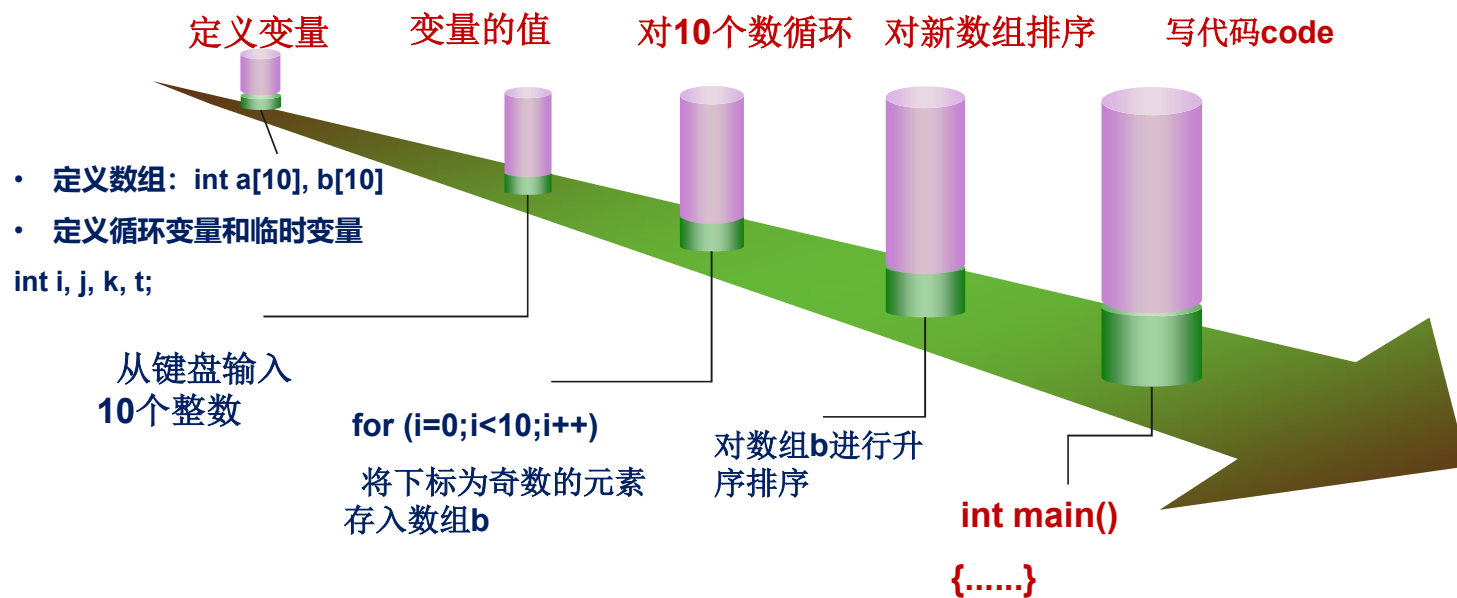
图形的形状特点是什么，上下对称？左右对称？边界上i与j的关系是什么？输入字符可能是大写或小写，遇到z如何处理。保密用例是什么？



编程题目总结

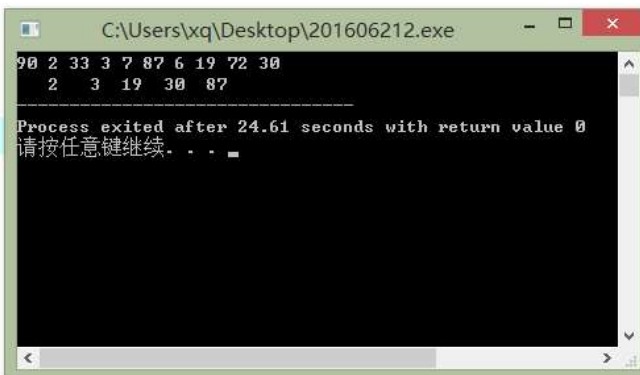
题目：从键盘输入10个数存入一个数组中，将下标为奇数的升序排序后输出

步骤：必须利用数组存放10个数，以及下标为奇数的数。



编程题目总结

```
1  #include<stdio.h>
2  int main()
3  { int a[10],b[10];
4    int i,j=0,k,t;
5    for(i=0;i<10;i++)
6      scanf("%d", &a[i]);
7    for(i=0;i<10;i++)
8      if(i%2!=0)
9        b[j++]=a[i];
10     k=j--;
11     for(i=1;i<k;i++)
12       for(j=0;j<k-i;j++)
13         if(b[j]>b[j+1])
14         {
15             t=b[j];b[j]=b[j+1];b[j+1]=t;
16         }
17     for(i=0;i<k;i++)
18       printf("%4d",b[i]) ;
19     return 0;
20 }
```



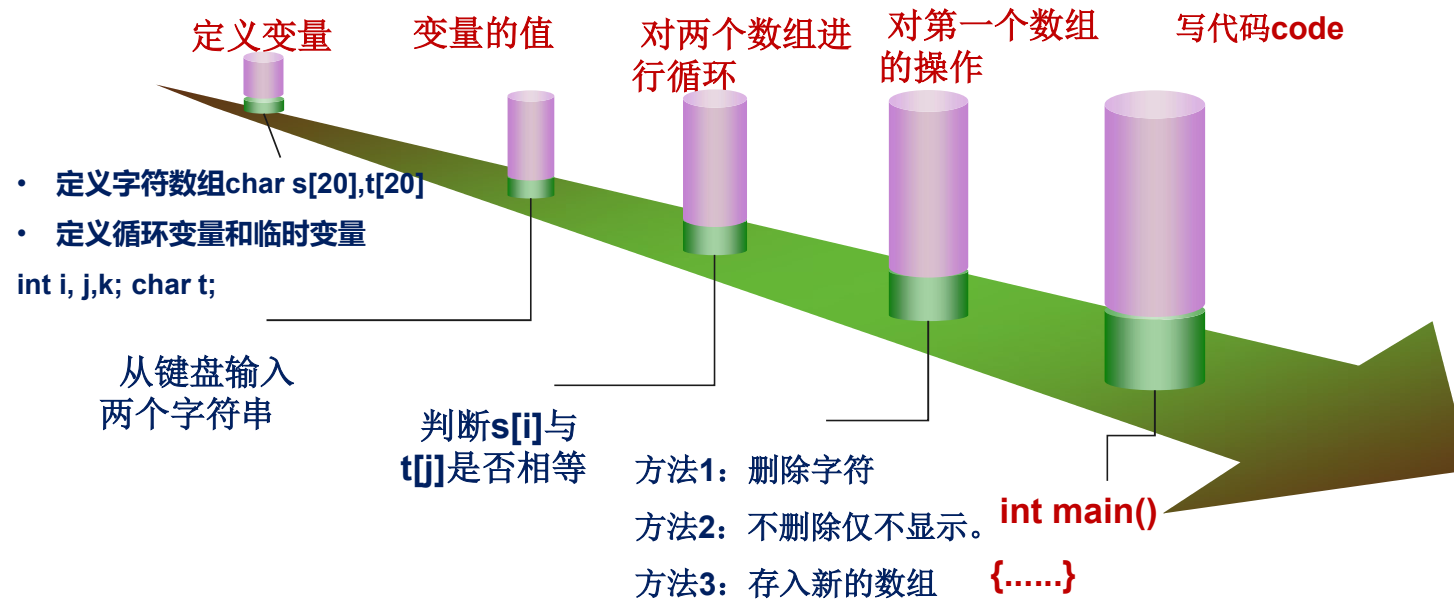
要点:

- 数组a下标为奇数的表示
- 数组b的下标变化
- 排序算法



编程题目总结

题目：从键盘输入两个字符串，从第一个串中删除第二个串中的字符后输出，
例如：输入第一个113245，第二个2367 输出：1145



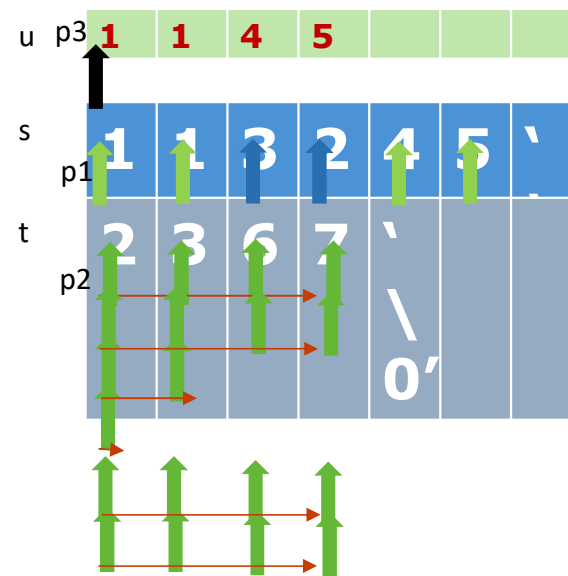
编程题目总结

程序1:

```
#include<stdio.h>
#include<string.h>
int main()
{ char s1[80],s2[80];
  int i,j,s,p,k=0;
  gets(s1);
  gets(s2);
  s=strlen(s1);
  p=strlen(s2);
  for(i=0;i<=s-1;i++)
  { for(j=0;j<=p-1;j++)
    { if(s1[i]==s2[j]) k++;
    }
    if(k==0) { printf("%c",s1[i]); }
    k=0;
  }
  printf("\n");
}
```

程序2:

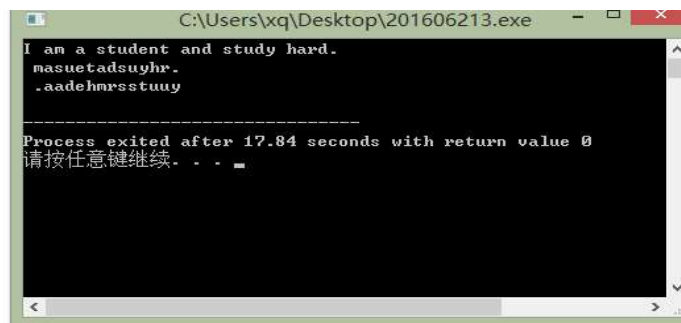
```
#include "stdio.h"
int main()
{ char s[80],t[80],u[80],*p1=s,*p2=t,*p3=u;
  gets(s);
  gets(t);
  while(*p1)
  { while(*p1!=*p2&&*p2)
    { p2++;
      if(!*p2)
        *p3++=*p1++;
      else p1++;
    }
    p2=t;
    *p3++='\0';
    puts(u);
    return 0;
  }
}
```



编程题目总结

题目：从键盘输入一个字符串，将下标为奇数的符号升序排序后输出

```
1 #include<stdio.h>
2 #include<stdio.h>
3 int main()
4 { char str1[20],str2[20];
5   int i,j=0,k;
6   char t;
7   gets(str1);
8   for(i=0;str1[i]!='\0' ;i++)
9     if(i%2!=0)
10      str2[j++]=str1[i];
11   puts(str2);
12   k=j--;
13   for(i=1;i<k&&str2[i]!='\0' ;i++)
14     for(j=0;j<k-i;j++)
15       if(str2[j]>str2[j+1])
16       { t=str2[j];str2[j]=str2[j+1];str2[j+1]=t;
17       }
18   str2[k]='\0';
19   puts(str2);
20   return 0;
21 }
```



要点:

- 字符数组输入输出
- 字符串处理函数，头文件
- 字符串结束标记
- 排序算法（改进：写一个函数）



部分练习

数制转换方面解题技巧:

- 认真转换。
- 选择题排除掉不可能选项后用按权展开法。

例：十进制数113转换成二进制数是（ ）。

A. 1110001

B. 1000111

C. 1110000

D. 10110000

排除C、D（结果为偶数）

用按权相加法 $A = 64 + 32 + 16 + 1 = 113$

A



部分练习

不是操作系统功能的是（ ）。

A.设备管理

B.信息管理

C.编译

D.存储管理

C



部分练习

【1】 以下不等式中，不正确的是()。

- A. $(1101010101010)_B > (FFF)_H$
- B. $(123456)_D < (123456)_H$
- C. $(1111)_D > (1111)_B$
- D. $(9)_H > (9)_D$

答案：D

【2】 8位反码表示的最大的数是()。

- A. 127
- B. 128
- C. 255
- D. 256

答案：C



部分练习

【3】二进制数10111000和11001010进行逻辑“与”运算，结果再与10100110进行逻辑“或”运算，最终结果的二进制形式为()。

- A. 11101011 B. 11101010
C. 10101110 D. 11010110

答案: C

【4】用8位二进制表示-23的补码是()。

答案: **11101001**

【5】补码1000 0000的真值是()。(十六进制)

答案: **-80H**

【6】8位补码00011001与11100000相加的结果为()。(十进制)。

答案: **-7**



部分练习

【7】一个C语言程序是由()组成的。

- A) 主程序 B) 子程序 C) 函数 D) 过程

答案：C

【8】C语言中char型数据在内存中是以()形式存储。

- A) 原码 B) 补码 C) 反码 D) ASCII码

答案：D

【9】C的运算符按运算对象的数目可以分为()。

- A) 单目运算符一种 B) 单目和双目运算符
C) 单目、双目和三目运算符
D) 单目、双目、三目和四目运算符

答案：C



部分练习

【10】在C语言中，错误的int类型的常数是 。(int默认2字节)

- A) 32768 B) 0 C) 037 D) 0xAF

答案：A



【11】以下选项中属于C语言的数据类型是 。

- A) 复数型 B) 逻辑型 C) 双精度型 D) 集合型

答案：C

【12】设int类型的数据长度为2个字节，则unsigned int类型数据的取值范围是 。

- A) 0至255 B) 0至65535
C) -32768至32767 D) -256至255

答案：B



部分练习

【13】已知：char w; int x; float y; double z; 则表达式 $w*x+z-y$ 的结果为()类型。

- A) float B) char C) int D) double

答案：D

【14】下列可以正确表示字符型常数的是()。

- A) "a" B) '\t' C) "\n" D) 297

答案：B

【15】已知：int a, b; 使用语句：scanf("%d%d", &a, &b); 输入a、b的值时，不能作为输入分隔符的是()。

- A) , B) 空格 C) 回车 D) Tab

答案：A



部分练习

【16】若给定条件表达式“(n)?(c++):(c--)”，则表达式n与表达式()是等价的。

- A) $n==0$ B) $n==1$ C) $n!=0$ D) $n!=1$

答案：C

【17】下列运算符的操作数必须是int型的运算是()。

- A) % B) / C) * D) &

答案：A

【18】已知：char *s= “\t\” Name\\Address\n “；则s所指的字符串的长度为()。

- A) 19 B) 15 C) 18 D) 说明不合法

答案：B



部分练习

【19】 为判断char型变量m是否是数字，可用表达式()。

- A) $0 \leq m \ \&\& \ m \leq 9$
- B) $'0' \leq m \ \&\& \ m \leq '9'$
- C) $"0" \leq m \ \&\& \ m \leq "9"$
- D) 前面三个答案均是错误的

答案： B



【20】 C对嵌套if的规定是：else总是与()配对。

- A) 其之前最近的if
- B) 第一个if
- C) 缩进位置相同的if
- D) 其之前最近的且尚未配对的if

答案： D



部分练习

【21】若 `int i=10`，则后，变量 `i` 的结果是()。

```
switch (i) {  
    case 9: i+=1;  
    case 10: i+=1;  
    case 11: i+=1;  
    default: i+=1;  
}
```

- A) 10 B) 11 C) 12 D) 13

答案: D

【22】若: `int a[3][4]`；则数组元素的非法引用是()。

- A) `a[0][2*1]` B) `a[1][3]`
C) `a[4-2][0]` D) `a[0][4]`

答案: D



部分练习

【23】 以下4个关于C语言的结论中，错误的是()。

- A) 用while语句实现的循环一定可用for语句实现；
- B) 用for语句实现的循环一定可用while语句实现；
- C) 用do-while语句实现的循环一定可用while语句实现；
- D) do-while语句与while语句的区别仅是关键字“while”出现的位置不同。



答案：D



部分练习

【24】若a、b均为int型变量且a=100，则对以下for循环语句的正确判断是()。

```
for (b=100; a!=b; ++a, b++)  
    printf ("----\n");
```

- A) 循环体只执行一次
- B) 是死循环
- C) 循环体一次也不执行
- D) 输出----

答案：C

【25】在C源程序文件中，若要定义一个不允许其他文件使用的变量，则该变量需要使用的存储类别是()。

- A) extern
- B) register
- C) auto
- D) static

答案：D

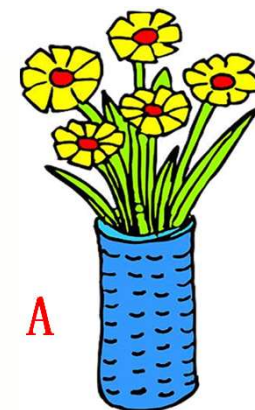


部分练习

【26】若用数组名作为函数调用时的实参，则传递给形参的是()。

- A) 数组首地址
- B) 数组的第一个元素值
- C) 数组中全部元素的值
- D) 数组元素的个数

答案：A



【27】执行下列语句后, i的值是()。

```
int i=1;  
for(j=1; j<(012^0x16); j++) i++;
```

A) 27 B) 28 C) 29 D) 30

答案：B



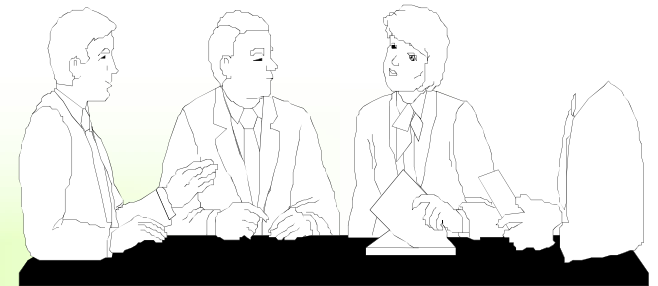
部分练习

【28】 已知函数定义如下：

```
void fun(char *s1, char *s2 )  
{  
    while ( *s2++ = *s1++ ) ;  
}
```

函数 fun 的功能是()。

- A) 串复制 B) 求串长 C) 串比较 D) 串反向



答案：A



部分练习

【29】下面的程序是求数组中的最小元素，请填空。

```
void findmin(int *s, int t, int *k) {  
    int p;  
    for ( p=0, *k=p; p<t; p++ )  
        if ( s[p] < s[*k] )  
            ① ;  
}  
  
int main() {  
    int a[10], i, *k = &i;  
    for (i=0; i<10; i++) scanf ("%d", &a[i]);  
    findmin (a, 10, k);  
    printf ("%d,%d\n", *k, a[*k]);  
    return 0;  
}
```

① *k = p



部分练习

【30】 下面的函数是求1到n之和。请填空。

```
int add ( int x )  
{  
    if ( n == 1 ) ①;  
    else return ②;  
}
```

答案: ① return 1
② n + add(n-1)



部分练习

【31】 程序读入20个整数，统计非负数个数，计算非负数之和。

```
int main() {  
    int i, a[20], s, count;  
    s=count=0;  
    for ( i=0; i<20; i++ )  
        scanf("%d", _____ ① );  
    for ( i=0; i<20; i++ ) {  
        if ( a[i]<0 )  
            _____ ② ;  
        s += a[i];  
        count++;  
    }  
    printf ("s=%d\t count=%d\n", s, count);  
    return 0;  
}
```



答案：① **&a[i]**
② **continue**



北京理工大学计算机学院
School of Computer Science and Technology, BIT

部分练习

【32】下面的函数fun的功能是将形参x的值转换成二进制数，所得二进制数的每一位放在一维数组中返回，二进制的最低位放在下标为0的元素中，其它依次类推。

```
void fun( int x, int b[ ] )
{
    int k=0, r;
    do
    {
        r = x % ①;
        b [k++] = r;
        x /= ②;
    } while (x);
}
```



答案：① 2
② 2

